

PART VI**SEARCHING AND SORTING****Extended Edition**

Every classic sort from first principles, and binary search
taken as far as it goes

You already call `sort()` and `binary_search()` from Part IV without a second thought. This Part goes underneath those calls: how each classic sorting algorithm actually works, why `std::sort` is safe to trust blindly, and — the real payoff — binary search pushed well past 'find a value in a sorted array' into the general skill of searching an answer space directly. This last skill alone unlocks a large fraction of medium/hard Codeforces and LeetCode problems.

Contents of Part VI

- 1. Sorting: The Big Picture
- 2. Bubble Sort
- 3. Selection Sort
- 4. Insertion Sort
- 5. Merge Sort
- 6. Quicksort
- 7. Heap Sort (Preview)
- 8. Counting Sort and Radix Sort
- 9. Custom Sorting and Why `std::sort` Wins in Practice
- 10. Linear Search
- 11. Binary Search, Deep Dive
- 12. Binary Search on the Answer
- 13. Recognizing a Binary-Search Problem
- Part VI Revision, Cheat Sheet, and Practice Drills

Chapter 1 — Sorting: The Big Picture

1.1 Why so many sorting algorithms?

In real competitive code you will call `sort()` and never write a sorting algorithm by hand. So why study eight of them in depth? Because sorting is the cleanest possible setting to practice everything Part V just formalized — best/average/worst case, stability, in-place vs. extra memory, and the recursion tree/Master Theorem machinery — on algorithms simple enough to fully understand in one sitting. The habits you build analyzing these eight algorithms transfer directly to analyzing your own solutions later.

1.2 The five properties every sort will be scored on

Property	Question it answers
Time complexity	Best / average / worst case, in Big-O
Space complexity	In-place ($O(1)$ extra) or does it need $O(n)$ extra memory?
Stability	Do equal elements keep their original relative order?
Adaptivity	Does it run faster on already-mostly-sorted input?
Practical use	When would a real competitive programmer reach for it over <code>std::sort</code> ?

NOTE — Why stability matters in practice

Stability matters whenever you sort by one key after already having sorted (or naturally ordered) by another — e.g. sort a list of students by grade, wanting ties broken by their original alphabetical order. An unstable sort can silently scramble that tie-breaking order, which is a real (if subtle) correctness bug, not just a cosmetic one.

Chapter 2 — Bubble Sort

2.1 Intuition

In one sentence: repeatedly walk through the array, swapping any two adjacent elements that are out of order, until nothing needs swapping — the largest unsorted element "bubbles" to its correct position each pass.

2.2 Implementation

```
void bubbleSort(vector<int> &a) {  
    int n = a.size();  
    for (int i = 0; i < n - 1; i++) {  
        bool swapped = false;  
        for (int j = 0; j < n - 1 - i; j++) {  
            if (a[j] > a[j + 1]) {  
                swap(a[j], a[j + 1]);  
                swapped = true;  
            }  
        }  
        if (!swapped) break; // already sorted – stop early  
    }  
}
```

2.3 Dry run: [5, 1, 4, 2]

Pass	Comparisons/swaps	Array after pass
1	5>1 swap, 5>4 swap, 5>2 swap	[1, 4, 2, 5]
2	1<4 no swap, 4>2 swap	[1, 2, 4, 5]
3	1<2 no swap — no swaps this pass, stop early	[1, 2, 4, 5]

2.4 Complexity and properties

Best	Average	Worst	Space	Stable?	In-place?
$O(n)$ (already sorted, early exit)	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes

WARNING — When (not) to use bubble sort

Bubble sort is essentially never the right choice in real code — it's taught purely for its simplicity as a first example. Its only redeeming practical property is the $O(n)$ best case on already-sorted input, which insertion sort (Chapter 4) also achieves while generally outperforming it on other inputs too.

Chapter 3 — Selection Sort

3.1 Intuition

In one sentence: repeatedly find the minimum of the remaining unsorted portion and swap it into its correct position at the front.

3.2 Implementation

```
void selectionSort(vector<int> &a) {
    int n = a.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIdx]) minIdx = j;
        swap(a[i], a[minIdx]);
    }
}
```

3.3 Dry run: [5, 1, 4, 2]

i	Minimum found in a[i..n-1]	Array after swap
0	1 at index 1	[1, 5, 4, 2]
1	2 at index 3	[1, 2, 4, 5]
2	4 at index 2 (already there)	[1, 2, 4, 5]

3.4 Complexity and properties

Best	Average	Worst	Space	Stable?	In-place?
$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No (naive version)	Yes

NOTE — Selection sort's one advantage

Unlike the other $O(n^2)$ sorts, selection sort makes at most $n-1$ swaps total, regardless of the input — every other comparison-based sort here can perform far more swaps. This makes it occasionally relevant when a swap (or write) is much more expensive than a comparison, e.g. sorting data on very slow storage.

Chapter 4 — Insertion Sort

4.1 Intuition

In one sentence: build the sorted array one element at a time, inserting each new element into its correct position among the already-sorted elements before it — exactly how most people sort a hand of playing cards.

4.2 Implementation

```

void insertionSort(vector<int> &a) {
    int n = a.size();
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j]; // shift larger elements right
            j--;
        }
        a[j + 1] = key;
    }
}

```

4.3 Dry run: [5, 1, 4, 2]

i	key	Shifting	Array after this i
1	1	5 > 1, shift 5 right	[1, 5, 4, 2]
2	4	5 > 4, shift 5 right; 1 < 4, stop	[1, 4, 5, 2]
3	2	5 > 2 shift, 4 > 2 shift, 1 < 2 stop	[1, 2, 4, 5]

4.4 Complexity and properties

Best	Average	Worst	Space	Stable?	In-place?
O(n) (already sorted)	O(n ²)	O(n ²)	O(1)	Yes	Yes

KEY IDEA — Where insertion sort is genuinely still used

Its $O(n)$ best case on nearly-sorted data, and excellent performance on very small arrays (fewer comparisons/overhead than the recursive sorts below), is exactly why real-world sort implementations — including C++'s introsort — switch to insertion sort for small subarrays once recursive splitting gets small enough.

Chapter 5 — Merge Sort

5.1 Intuition

In one sentence: split the array in half, recursively sort each half, then merge the two sorted halves back together in linear time — the textbook divide-and-conquer algorithm.

5.2 Implementation

```

void merge(vector<int> &a, int l, int m, int r) {
    vector<int> left(a.begin() + l, a.begin() + m + 1);
    vector<int> right(a.begin() + m + 1, a.begin() + r + 1);
    int i = 0, j = 0, k = l;
    while (i < (int)left.size() && j < (int)right.size()) {
        if (left[i] <= right[j]) a[k++] = left[i++]; // <= keeps it
        stable
        else a[k++] = right[j++];
    }
    while (i < (int)left.size()) a[k++] = left[i++];
    while (j < (int)right.size()) a[k++] = right[j++];
}

void mergeSort(vector<int> &a, int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}

```

WARNING — Use $l + (r-l)/2$, not $(l+r)/2$

For very large l and r , $(l+r)$ can overflow before the division happens (Part I, Section 3.4). $l + (r-l)/2$ computes the same midpoint without ever letting the sum exceed either bound — a habit worth having permanently for any midpoint calculation, including binary search later in this Part.

5.3 Dry run: recursion tree for [5, 1, 4, 2]

Call	Splits into	Result after merge
mergeSort([5,1,4,2])	[5,1] and [4,2]	—
mergeSort([5,1])	[5] and [1]	merge -> [1,5]
mergeSort([4,2])	[4] and [2]	merge -> [2,4]
Top-level merge	[1,5] and [2,4]	merge -> [1,2,4,5]

5.4 Complexity via the Master Theorem

As derived in Part V, Chapter 4: $a=2$ subproblems, $b=2$ (each half the size), $d=1$ (merging is $O(n)$). $\log_b(a) = 1 = d \rightarrow$ the balanced case $\rightarrow T(n) = O(n \log n)$, in the best, average, AND worst case — merge sort's performance never degrades, unlike quicksort's.

Best	Average	Worst	Space	Stable?	In-place?
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No

NOTE — Why merge sort needs $O(n)$ extra space

The merge step needs to read from both halves while writing the combined result — doing this fully in-place ($O(1)$ extra space) is possible but significantly more complex and slower in practice; the standard implementation always allocates temporary arrays as shown above.

Chapter 6 — Quicksort

6.1 Intuition

In one sentence: pick a pivot element, partition the array so everything smaller than the pivot is on its left and everything larger is on its right, then recursively sort each side.

6.2 Implementation (Lomuto partition scheme)

```

int partition(vector<int> &a, int lo, int hi) {
    int pivot = a[hi];
    int i = lo - 1;
    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[hi]);
    return i + 1; // final index of the pivot
}

void quickSort(vector<int> &a, int lo, int hi) {
    if (lo >= hi) return;
    int p = partition(a, lo, hi);
    quickSort(a, lo, p - 1);
    quickSort(a, p + 1, hi);
}

```

6.3 Dry run: partition([5,1,4,2], pivot = 2)

j	a[j]	Action	Array state
0	5	5 >= pivot(2), no swap	[5,1,4,2]
1	1	1 < 2 -> i=0, swap a[0],a[1]	[1,5,4,2]
2	4	4 >= 2, no swap	[1,5,4,2]
end	—	swap a[i+1]=a[1] with a[hi]=a[3]	[1,2,4,5], pivot now at index 1

6.4 Complexity and the pivot-choice problem

Best / Average	Worst	Space (recursion stack)	Stable ?	In-place?
$O(n \log n)$	$O(n^2)$	$O(\log n)$ avg, $O(n)$ worst	No	Yes

Worst case happens when the pivot is consistently the smallest or largest element — for the naive "pivot = last element" scheme above, this happens on already-sorted (or reverse-sorted) input, producing maximally unbalanced partitions of size 0 and $n-1$ at every level, giving $O(n)$ levels of $O(n)$ work each: $O(n^2)$.

WARNING — Why hand-rolled quicksort has been exploitable on judges

If your pivot choice is deterministic and known (e.g. always the first or last element), an adversarial problem setter can construct input specifically to trigger the $O(n^2)$ worst case, causing TLE. Mitigations: choose a random pivot, or the median of three candidates, before partitioning. This is precisely why competitive programmers default to `std::sort` (Chapter 9) rather than hand-rolled quicksort.

Chapter 7 — Heap Sort (Preview)

Heap sort's underlying structure (the binary heap) is covered fully in Part XII; this preview covers enough to complete the sorting-algorithm comparison here.

7.1 Intuition

In one sentence: build a max-heap from the array, then repeatedly remove the maximum (swap it to the end, shrink the heap by one, and restore the heap property) until the array is sorted.

```

void heapify(vector<int> &a, int n, int i) {
    int largest = i, l = 2*i + 1, r = 2*i + 2;
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) {
        swap(a[i], a[largest]);
        heapify(a, n, largest);
    }
}

void heapSort(vector<int> &a) {
    int n = a.size();
    for (int i = n / 2 - 1; i >= 0; i--) heapify(a, n, i); // build
max-heap: O(n)
    for (int i = n - 1; i > 0; i--) {
        swap(a[0], a[i]); // move current max to the end
        heapify(a, i, 0); // restore heap property on the shrunk
heap
    }
}

```

Best	Average	Worst	Space	Stable?	In-place?
$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes

KEY IDEA — Heap sort's role in `std::sort`

Heap sort guarantees $O(n \log n)$ worst case AND $O(1)$ extra space — the combination quicksort lacks (needs randomization to guarantee $O(n \log n)$) and merge sort lacks (needs $O(n)$ space). This is exactly why introsort (Chapter 9) falls back to heap sort specifically when quicksort's recursion depth suggests a worst-case pattern is happening.

Chapter 8 — Counting Sort and Radix Sort

8.1 Counting sort: sorting without comparisons

Every sort so far has compared pairs of elements, and it's a proven fact that any comparison-based sort needs at least $O(n \log n)$ comparisons in the worst case. Counting sort sidesteps comparisons entirely when values are integers within a known, small range $[0, k]$: count how many times each value appears, then reconstruct the sorted array directly from those counts.

```
vector<int> countingSort(vector<int> &a, int k) { // values in [0, k]
    vector<int> count(k + 1, 0);
    for (int x : a) count[x]++;
    vector<int> result;
    for (int v = 0; v <= k; v++)
        for (int c = 0; c < count[v]; c++) result.push_back(v);
    return result;
}
```

Best/Average/Worst	Space	Stable?	Precondition
$O(n + k)$	$O(n + k)$	Yes (with the standard prefix-sum variant)	Values must be small non-negative integers

WARNING — Counting sort is only fast when k isn't too large

$O(n + k)$ is only better than $O(n \log n)$ when $k = O(n)$ or smaller. Sorting values up to 10^9 with counting sort would need a count array of size 10^9 — far too much memory. Check the value range against n before reaching for this technique.

8.2 Radix sort (concept)

Radix sort sorts multi-digit numbers by repeatedly applying a stable counting sort on one digit position at a time, from the least significant digit to the most significant. Because each pass is a stable $O(n + \text{base})$

counting sort and there are $O(\log_{\text{base}}(\text{max_value}))$ digit positions, the total complexity is $O(d \cdot (n + \text{base}))$ where d is the number of digits — effectively $O(n)$ for fixed-size integers, avoiding the $O(n \log n)$ comparison lower bound entirely.

NOTE — Where these show up in competitive programming

Both are used rarely as a whole sort in practice (`std::sort` is fast enough and far less code), but the underlying idea — bucket values by a small key rather than comparing them — reappears in bucket-based approaches to problems like sorting by frequency, or as a building block in specialized suffix-array construction algorithms (beyond this book's scope).

Chapter 9 — Custom Sorting and Why `std::sort` Wins in Practice

9.1 Introsort: the best of three worlds

GCC's `std::sort` uses introsort: start with quicksort for its excellent average-case speed and cache behaviour; if the recursion depth exceeds a threshold (roughly $2 \cdot \log_2(n)$, a signal that the partitioning is degenerating toward the $O(n^2)$ worst case), switch to heap sort to guarantee $O(n \log n)$; and switch to insertion sort once a subarray shrinks below a small threshold (typically around 16 elements), since insertion sort has less overhead than recursive calls for tiny inputs. The result: $O(n \log n)$ worst-case guarantee, excellent average-case constant factor, and no vulnerability to the classic adversarial-input attack on naive quicksort.

KEY IDEA — The practical takeaway from this entire chapter

This is why every earlier Part in this book has said, without much justification, "just use `sort()`." You now know exactly why that advice is safe: introsort inherits quicksort's speed, heap sort's worst-case guarantee, and insertion sort's small-input efficiency, and hand-rolling any single one of Chapters 2-7 in a contest would almost always be strictly worse.

9.2 Recap: custom comparators (from Part IV)

```
sort(v.begin(), v.end(), greater<int>()); // descending
sort(v.begin(), v.end(), [](int a, int b){ return a > b; }); //
equivalent, via lambda

sort(people.begin(), people.end(), [](auto &a, auto &b) {
    if (a.first != b.first) return a.first < b.first;
    return a.second > b.second;
});
```

Chapter 10 — Linear Search

10.1 The baseline

```
int linearSearch(vector<int> &a, int target) {
    for (int i = 0; i < (int)a.size(); i++)
        if (a[i] == target) return i;
    return -1;
}
```

$O(n)$ worst case, works on any array, sorted or not. This is the correct choice whenever the array isn't sorted and won't be searched many times (sorting purely to enable one binary search costs $O(n \log n)$, more than a single $O(n)$ linear scan).

Chapter 11 — Binary Search, Deep Dive

11.1 The core algorithm

In one sentence: on a sorted array, repeatedly halve the search range by checking the middle element, eliminating half the remaining possibilities every time.

```
int binarySearch(vector<int> &a, int target) {  
    int lo = 0, hi = (int)a.size() - 1;  
    while (lo <= hi) {  
        int mid = lo + (hi - lo) / 2;  
        if (a[mid] == target) return mid;  
        else if (a[mid] < target) lo = mid + 1;  
        else hi = mid - 1;  
    }  
    return -1; // not found  
}
```

Dry run: a = [1,3,5,7,9,11], target = 7

lo	hi	mid	a[mid]	Action
0	5	2	5	5 < 7 -> lo = 3
3	5	4	9	9 > 7 -> hi = 3
3	3	3	7	found! return 3

WARNING — Why binary search is notoriously easy to get subtly wrong

Off-by-one errors in the loop condition ($\leq$ vs $<$) and the update step (mid vs $\text{mid} \pm 1$) are the single most common source of bugs in hand-written binary search. Using the exact template above consistently — $\text{lo} \leq \text{hi}$, $\text{mid}+1/\text{mid}-1$ on the branches, $\text{l}+(\text{r}-\text{l})/2$ for the midpoint — removes almost all of this risk. Re-deriving binary search from scratch under time pressure is a common source of wrong answers; memorize one correct template instead.

11.2 First and last occurrence in an array with duplicates

```
int firstOccurrence(vector<int> &a, int target) {
    int lo = 0, hi = (int)a.size() - 1, result = -1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] == target) { result = mid; hi = mid - 1; } // keep
        searching left
        else if (a[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return result;
}
// lastOccurrence is symmetric: on a match, set result and search
// RIGHT (lo = mid + 1) instead.
```

This is precisely what `lower_bound/upper_bound` (Part IV) do internally — implementing it by hand once here cements exactly what those library calls are doing underneath.

11.3 Reimplementing `lower_bound` and `upper_bound`

```
int myLowerBound(vector<int> &a, int target) { // first index with
a[i] >= target
    int lo = 0, hi = (int)a.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] < target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

int myUpperBound(vector<int> &a, int target) { // first index with
a[i] > target
    int lo = 0, hi = (int)a.size();
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] <= target) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

NOTE — A second valid template

Notice this version uses a half-open range $[lo, hi)$ with $lo < hi$ as the loop condition, rather than the closed $[lo, hi]$ with $lo \leq hi$ from Section 11.1. Both are correct, well-known templates — pick one, understand it fully, and use it consistently rather than mixing conventions mid-problem.

Chapter 12 — Binary Search on the Answer

12.1 The generalization

Binary search doesn't require a literal sorted array — it only requires a monotonic predicate: a yes/no question about a candidate answer x , such that once the answer is "yes" for some x , it stays "yes" for every

value beyond x in the same direction (or symmetrically for "no"). When this holds, you can binary-search directly over the space of possible answers, checking each candidate with a "can we achieve this?" function, rather than searching over array indices.

KEY IDEA — The three ingredients of binary search on the answer

- 1 A search space of candidate answers with a clear minimum and maximum (e.g. 1 to 10^9).
- 2 A monotonic feasibility check: $\text{can}(x)$ — is x achievable, and does the answer stay achievable for every value on one side of x ?
- 3 A binary search driving that check toward the boundary between 'achievable' and 'not achievable'.

12.2 Worked example: minimum capacity to ship packages within D days

Problem: given package weights and a number of days D , packages must be shipped in the given order, each day carrying a contiguous run of packages up to a ship capacity limit. Find the minimum capacity that lets everything ship within D days.

Applying the framework:

- 1 Search space: capacity ranges from $\max(\text{weights})$ (must fit the single heaviest package) to $\text{sum}(\text{weights})$ (fits everything in one day).
- 2 Feasibility check: $\text{can}(\text{capacity})$ — simulate shipping greedily with this capacity, counting how many days it takes; return true if that count is $\leq D$.
- 3 Monotonicity: if capacity C works (needs $\leq D$ days), any larger capacity also works, since more room per day can only reduce or maintain the day count — never increase it. This justifies binary search.

```
bool canShipWithinDays(vector<int> &weights, int capacity, int D) {
    int days = 1, currentLoad = 0;
    for (int w : weights) {
        if (currentLoad + w > capacity) { days++; currentLoad = 0; }
        currentLoad += w;
    }
    return days <= D;
}

int minCapacity(vector<int> &weights, int D) {
    int lo = *max_element(weights.begin(), weights.end());
    int hi = accumulate(weights.begin(), weights.end(), 0);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canShipWithinDays(weights, mid, D)) hi = mid; // mid
works - try smaller
        else lo = mid + 1; // mid too small - need more
    }
    return lo;
}
```

Complexity

The feasibility check is $O(n)$; binary search over the capacity range runs $O(\log(\text{sum of weights}))$ times: $O(n \log(\text{sum}))$ total — turning what looks like an optimization problem into a search problem an order of magnitude simpler than it first appears.

12.3 Worked example: the classic "minimum feasible value" phrasing

A large family of problems is phrased as "find the minimum/maximum X such that holds." Whenever the condition is monotonic in X , this phrasing is a direct invitation to binary search on the answer, even when the problem never mentions searching or sorting at all.

Problem phrasing	What you binary search over	Feasibility check
Minimum capacity to finish in D days	Capacity value	Simulate shipping, count days needed
Minimum time for all workers to finish a task	Time value	Can all work be completed by this time?
Maximum minimum distance between placed items (e.g. cows in stalls)	Distance value	Can items be placed with at least this much spacing?
Smallest divisor such that a sum of ceil-divisions stays under a threshold	Divisor value	Compute the sum with this divisor; is it under the threshold?

Chapter 13 — Recognizing a Binary-Search Problem

13.1 Clue words

- "Minimum/maximum X such that..." — especially combined with a simulate-and-check structure.
- "Find the smallest/largest value for which a condition holds."
- A problem where trying every candidate answer directly would be too slow, but checking a single candidate is fast and the check is monotonic.
- A sorted array combined with any 'find the boundary between two regions' question — first true, last true, first occurrence, insertion point.

13.2 The single question that confirms it

KEY IDEA — Ask this before committing to binary search on the answer

"If x works, does $x+1$ (or $x-1$, depending on direction) also work?" If you can answer yes with a clear justification (as in Section 12.2's shipping-capacity argument), monotonicity holds and binary search on the answer is valid. If you cannot justify this — if working and not-working values can be interleaved unpredictably — binary search on the answer will silently give a wrong answer, so do not apply it without this check.

Problem 13.1 — Recognize and outline [Level 3]

Problem: given n piles of bananas and h hours, an eating speed k means eating k bananas per hour from one pile per hour (finishing a pile early wastes the rest of that hour). Find the minimum integer eating speed k so that all piles are finished within h hours.

Hint 1: What are you actually searching over — pile values, or something else entirely?

Key observation: If speed k finishes in time, any faster speed also finishes in time or faster — monotonic in k .

Approach: Binary search k between 1 and $\max(\text{piles})$. Feasibility check: sum of $\text{ceil}(\text{pile} / k)$ over all piles, compare against h .

Complexity: $O(n \log(\max(\text{piles})))$ — each feasibility check is $O(n)$, binary search adds a log factor.

Part VI — Master Revision

Must Remember

- Always use $l + (r-l)/2$ for a midpoint calculation, never $(l+r)/2$, to avoid overflow.
- `std::sort` (introsort) combines quicksort's speed, heap sort's $O(n \log n)$ worst-case guarantee, and insertion sort's small-input efficiency — hand-rolling a sort is almost never a good idea in a contest.
- Merge sort is always $O(n \log n)$ but needs $O(n)$ extra space; quicksort is in-place but needs randomization to avoid an $O(n^2)$ adversarial worst case.
- Counting/radix sort beat the $O(n \log n)$ comparison-sort lower bound, but only when the value range k is small enough that $O(n + k)$ is actually better.
- Binary search generalizes far beyond arrays: any monotonic yes/no predicate over a range of candidate answers can be binary-searched directly (binary search on the answer).
- Before applying binary search on the answer, explicitly justify monotonicity: 'if x works, does the next value in the same direction also work?'

Complexity and Properties — Master Table

Algorithm	Best	Avg	Worst	Space	Stable
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes

Common Mistakes Recap

Mistake	Fix
Computing $\text{mid} = (\text{lo} + \text{hi})/2$	Use $\text{lo} + (\text{hi} - \text{lo})/2$ to avoid overflow
Deterministic (first/last element) pivot in a hand-rolled quicksort	Randomize the pivot, or just use <code>std::sort</code>
Applying binary search on the answer without checking monotonicity	Explicitly justify 'if x works, does $x+1$ also work' first
Using counting sort on a huge value range	Check that $k = O(n)$; otherwise use a comparison sort
Mixing the closed $[\text{lo}, \text{hi}]$ and half-open $[\text{lo}, \text{hi})$ binary search templates mid-solution	Pick one template and use it consistently

Interview-Style Questions

- 1 Why does merge sort guarantee $O(n \log n)$ in every case, while quicksort does not?
- 2 Explain introsort's three-algorithm strategy and why each piece is chosen for its specific role.

- 3 Why is counting sort not a counterexample to the 'comparison sorts need $O(n \log n)$ ' lower bound?
- 4 What three ingredients justify applying binary search on the answer to a new problem?
- 5 Why is $l + (r-l)/2$ preferred over $(l+r)/2$ for computing a midpoint?

Cheat Sheet — Part VI

```
// Standard binary search template (closed range)
int lo = 0, hi = n - 1;
while (lo <= hi) {
    int mid = lo + (hi - lo) / 2;
    if (check(mid)) /* record answer if needed */ hi = mid - 1; // or
    lo = mid + 1, depending on goal
    else lo = mid + 1; // or hi = mid - 1
}

// Binary search on the answer (half-open range, find smallest
feasible value)
int lo = MIN_POSSIBLE, hi = MAX_POSSIBLE;
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid)) hi = mid;
    else lo = mid + 1;
}
// lo == hi is the smallest feasible value
```

NOTE — Coming up in Part VII

Part VII — Hashing goes deep on hash tables themselves: how `unordered_map` actually computes bucket indices, collision resolution strategies, writing a custom hash function (including the anti-hash-hacking defense mentioned in Part IV), and the frequency-map and two-sum-style patterns that recur across a large fraction of easy-to-medium problems.